# CSA 1737 – ARTIFICIAL INTELLIGENCE

Laboratory Record – Experiments 16 to 20

## Ex. No. 16: Breadth First Search (BFS)

### AIM

To write a Python program to implement the Breadth First Search (BFS) graph traversal algorithm.

### ALGORITHM

Step 1: Start the program.

Step 2: Represent the graph as an adjacency list.

Step 3: Insert the starting node into a queue and mark it visited.

Step 4: Remove the front node from the queue and print it.

Step 5: Insert all unvisited neighbours of that node into the queue and mark them visited.

Step 6: Repeat steps 4 and 5 until the queue is empty.

Step 7: Stop the program.

### CODE

```
# Experiment 16: Breadth First Search (BFS)
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print(node, end=" ")
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("Graph (adjacency list):", graph)
print("BFS traversal starting from A:")
bfs(graph, 'A')
print()
```

### OUTPUT

```
Graph (adjacency list): {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [],
'E': ['F'], 'F': []}
BFS traversal starting from A:
A B C D E F
```

## RESULT

Thus the Python program to implement BFS was written and executed successfully, and the traversal order was verified.

# Ex. No. 17: Depth First Search (DFS)

## AIM

To write a Python program to implement the Depth First Search (DFS) graph traversal algorithm.

## ALGORITHM

Step 1: Start the program.

Step 2: Represent the graph as an adjacency list and create an empty visited set.

Step 3: Call dfs() with the starting node.

Step 4: If the node is not visited, print it and mark it visited.

Step 5: Recursively call dfs() for each neighbour of the node.

Step 6: Stop when all reachable nodes are visited.

Step 7: Stop the program.

## CODE

```
# Experiment 17: Depth First Search (DFS)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(graph, neighbour, visited)

print("Graph (adjacency list):", graph)
print("DFS traversal starting from A:")
dfs(graph, 'A', set())
print()
```

## OUTPUT

```
Graph (adjacency list): {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [],
'E': ['F'], 'F': []}
DFS traversal starting from A:
A B D E F C
```

## RESULT

Thus the Python program to implement DFS was written and executed successfully, and the traversal order was verified.

# Ex. No. 18: Travelling Salesman Problem

## AIM

To write a Python program to implement the Travelling Salesman Problem (TSP) and find the shortest route that visits every city once and returns to the start.

## ALGORITHM

Step 1: Start the program.

Step 2: Store the cities and the distance matrix.

Step 3: Fix city A as the starting and ending city.

Step 4: Generate all permutations of the remaining cities.

Step 5: For each permutation, build the route A -> ... -> A and calculate its total cost using the distance matrix.

Step 6: Keep track of the route with the minimum cost.

Step 7: Display the shortest route and the minimum cost.

Step 8: Stop the program.

## CODE

```
# Experiment 18: Travelling Salesman Problem (brute force)
from itertools import permutations

cities = ['A', 'B', 'C', 'D']
# Distance matrix (symmetric)
dist = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(cities)
start = 0                       # start and end at city A
best_cost = float('inf')
best_route = None

for perm in permutations(range(1, n)):
    route = (start,) + perm + (start,)
    cost = 0
    for i in range(n):
        cost += dist[route[i]][route[i + 1]]
    print("Route:", " -> ".join(cities[c] for c in route), "| Cost =", cost)
    if cost < best_cost:
        best_cost = cost
        best_route = route

print("\nShortest route :", " -> ".join(cities[c] for c in best_route))
print("Minimum cost   :", best_cost)
```

## OUTPUT

```
Route: A -> B -> C -> D -> A | Cost = 95
Route: A -> B -> D -> C -> A | Cost = 80
```

```
Route: A -> C -> B -> D -> A | Cost = 95
Route: A -> C -> D -> B -> A | Cost = 80
Route: A -> D -> B -> C -> A | Cost = 95
Route: A -> D -> C -> B -> A | Cost = 95

Shortest route : A -> B -> D -> C -> A
Minimum cost   : 80
```

## RESULT

Thus the Python program for the Travelling Salesman Problem was written and executed successfully; the shortest route A -> B -> D -> C -> A with cost 80 was found.

# Ex. No. 19: A* Algorithm

## AIM

To write a Python program to implement the A* search algorithm.

## ALGORITHM

Step 1: Start the program.

Step 2: Represent the weighted graph and the heuristic value h(n) of every node.

Step 3: Put the start node in the open list with f(n) = g(n) + h(n).

Step 4: Remove the node with the smallest f(n) from the open list.

Step 5: If it is the goal node, return the path and cost.

Step 6: Otherwise add it to the closed list and, for each neighbour, calculate g(n) and f(n) and insert it into the open list.

Step 7: Repeat steps 4 to 6 until the goal is found or the open list is empty.

Step 8: Stop the program.

## CODE

```
# Experiment 19: A* Algorithm
import heapq

# Weighted graph
graph = {
    'S': {'A': 1, 'B': 4},
    'A': {'B': 2, 'C': 5, 'D': 12},
    'B': {'C': 2},
    'C': {'D': 3, 'G': 3},
    'D': {'G': 3},
    'G': {}
}

# Heuristic values h(n) : estimated cost from node n to goal G
h = {'S': 7, 'A': 6, 'B': 4, 'C': 3, 'D': 3, 'G': 0}

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]     # (f, g, node, path)
    closed = set()
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node == goal:
            return path, g
        if node in closed:
            continue
        closed.add(node)
        for nxt, cost in graph[node].items():
            if nxt not in closed:
                g2 = g + cost
                heapq.heappush(open_list, (g2 + h[nxt], g2, nxt, path + [nxt]))
    return None, None

path, cost = a_star('S', 'G')
print("Shortest path found by A* :", " -> ".join(path))
print("Total cost                :", cost)
```

## OUTPUT

```
Shortest path found by A* : S -> A -> B -> C -> G
Total cost                : 8
```

## RESULT

Thus the Python program to implement the A* algorithm was written and executed successfully; the path S -> A -> B -> C -> G with cost 8 was found.

# Ex. No. 20: Map Colouring (Constraint Satisfaction Problem)

## AIM

To write a Python program for the Map Colouring problem, implemented as a Constraint Satisfaction Problem (CSP).

## ALGORITHM

Step 1: Start the program.

Step 2: Define the variables (regions of the map), the domain (colours Red, Green, Blue) and the constraint: adjacent regions must have different colours.

Step 3: Select an unassigned region.

Step 4: Try each colour; assign it if it does not conflict with the colours of the neighbouring regions.

Step 5: Recursively assign the next region.

Step 6: If no colour is possible, backtrack: remove the previous assignment and try another colour.

Step 7: When all regions are assigned, display the solution.

Step 8: Stop the program.

## CODE

```
# Experiment 20: Map Coloring as a Constraint Satisfaction Problem (CSP)
# Map of Australia
variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
colors = ['Red', 'Green', 'Blue']

neighbors = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V': ['SA', 'NSW'],
    'T': []
}

def is_consistent(region, color, assignment):
    for nb in neighbors[region]:
        if assignment.get(nb) == color:
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment
    region = [v for v in variables if v not in assignment][0]
    for color in colors:
        if is_consistent(region, color, assignment):
            assignment[region] = color
            result = backtrack(assignment)
            if result:
                return result
```

```
            del assignment[region]          # backtrack
    return None

solution = backtrack({})
if solution:
    print("Map colouring solution:")
    for region in variables:
        print(f"{region:4s} -> {solution[region]}")
else:
    print("No solution exists")
```

## OUTPUT

```
Map colouring solution:
WA   -> Red
NT   -> Green
SA   -> Blue
Q    -> Red
NSW  -> Green
V    -> Red
T    -> Red
```

## RESULT

Thus the Python program for Map Colouring using CSP was written and executed successfully, and no two adjacent regions have the same colour.